

面试作业：Web 自动化脚本生成 Agent

(借鉴 AgentBrowser 架构，底层使用 DrissionPage)

一、背景

AgentBrowser 是一个典型的 AI 浏览器代理框架，其核心设计包含三个紧密协作的模块：

- 生成模块 (Generation)：将自然语言任务分解为浏览器操作序列，并生成可执行的自动化脚本。
- Debug 模块 (Debugging)：自动运行脚本，发现失败后结合页面状态与错误信息，驱动 LLM 修复脚本，直至通过。
- 容错模块 (Resilience)：在录制与回放过程中，通过选择器降级、智能等待、异常处理等策略，提升脚本的鲁棒性。

本次作业要求借鉴 AgentBrowser 的上述三模块设计，但底层浏览器自动化库从 Playwright 替换为 DrissionPage (Python)，实现一个端到端的“自然语言 → 经过自我验证和修复的 DrissionPage 脚本”的 Agent 系统。

二、题目目标

使用 Python 和 DrissionPage，结合任意 LLM API (如 OpenAI)，实现一个 Web 自动化脚本生成 Agent。

该系统必须包含且解耦以下三个模块：

- 生成模块：输入自然语言，输出一份初版 DrissionPage 脚本。
- Debug 模块：自动执行初版脚本，捕获失败并调用 LLM 修复，循环直至脚本稳定通过。
- 容错模块：在生成与执行中，提供选择器降级、智能等待、异常处理等通用健壮性能力。

最终用户只需一句话，就能得到一个独立、可重复运行的 `.py` 脚本，该脚本不依赖任何 LLM 服务。

三、任务详情

1. 生成模块 (Generation)

- 操作分解与执行

利用 LLM 将自然语言任务分解为具体浏览器操作步骤 (导航、点击、输入、选择等)，并使用 DrissionPage (`ChromiumPage`) 逐步执行。

- 动作捕获

在执行过程中，记录每一步的详细动作信息，包括：

- 动作类型 (goto, click, input, select, upload, wait 等)。

- 元素定位信息：优先提取 `id`、`name`、`aria-label`、`data-*` 属性、可见文本等稳定特征，禁止仅使用脆弱的下标或长 XPath/CSS 路径。
- 动作数据（输入文本、选择项等）。
- 脚本生成
根据捕获的步骤，自动生成一份 DrissionPage Python 脚本，要求：
 - 使用捕获到的稳定选择器，结合 DrissionPage 定位语法，如 `ele('@id=xxx')`、`ele('text:登录')`、`ele('@name=username')` 等。
 - 包含显式等待（如 `wait.ele_displayed('text:搜索')`）、页面加载等待（`wait.load_start()`）等。
 - 保留清晰注释（例如 `# 点击搜索按钮`），代码可读性强。
 - 脚本保存为 `.py` 文件，独立于 LLM 和 Agent 可直接运行。

2. Debug 模块 (Debugging)

- 自动验证
自动执行生成模块产出的脚本，记录执行结果（成功/失败）和标准输出/错误。
- 失败诊断与修复
若脚本执行失败：
 - 收集异常信息（异常类型、错误消息、脚本哪一行失败）。
 - 获取当前页面部分 DOM 快照或关键元素状态（可通过 DrissionPage 的 `page.html` 截取片段）。
 - 将失败上下文和原始脚本提交给 LLM，让 LLM 分析原因并生成修复后的新脚本（如调整选择器、增加等待、修正操作顺序）。
- 修复循环
循环执行“执行 → 失败 → LLM修复 → 重新执行”的过程，直到：
 - 脚本成功执行完成；或
 - 达到最大重试次数（如 3 次）。
- 输出
 - 最终通过验证的脚本。
 - 简要修复日志（例如：“[Debug] 第1次失败：元素 `@id=searchBtn` 定位超时，已修改为 `text:搜索按钮`”）。

3. 容错模块 (Resilience)

该模块为生成模块和 Debug 模块提供可复用的健壮性能力，并最终体现在生成的脚本中：

- 选择器降级策略

定义选择器优先级（如：`id` > `name` > `aria-label` > 文本匹配 > 属性模糊匹配）。

当优先选择器定位失败时，自动尝试下一级选择器。生成的脚本应体现这一策略（例如使用循环尝试多个定位方式）。

- 智能等待与重试

避免固定 `time.sleep`，使用 DrissionPage 的 `wait.ele_displayed()`、`wait.load_complete()` 等。

对关键操作可配置重试次数（如点击后等待元素出现，若超时则重试一次点击）。

- 异常处理

生成的脚本必须用 `try...except` 包裹可能失败的操作，输出友好错误信息，而非静默崩溃。

- 动作抽象

将动作统一为可扩展的基类（如 `Action`），方便后续支持拖拽、Hover、键盘组合等，且容错逻辑可复用。

4. 系统整体要求

- 三个模块应低耦合、可独立测试，例如可单独调用容错模块的选择器降级函数，或单独测试 Debug 模块的修复逻辑。
- 提供至少 2 个完整的端到端示例，每个示例包含：
 - 输入的一句自然语言。
 - 生成模块产出的初版脚本。
 - Debug 模块的修复过程日志（如有失败）。
 - 最终稳定的 `.py` 脚本。
 - 脚本成功执行的截图或日志。
- 覆盖至少 4 种交互类型：页面导航、点击元素、文本输入、下拉选择（或文件上传等）。

5. 工程规范

- 代码结构清晰，建议划分为 `generation/`，`debug/`，`resilience/` 等模块。
- 编写详细的 `README.md`：
 - 环境要求（Python 版本、DrissionPage 版本等）。
 - 安装步骤：`pip install drissionpage openai` 等。
 - 配置说明（如环境变量 `OPENAI_API_KEY`）。
 - 各模块设计思路（生成流程、Debug 循环、容错策略）。
 - 使用方法和运行示例（命令行或 Python API）。
 - 示例执行截图或录屏链接。

- 可额外创新：自动生成断言（验证页面标题、元素文本）、操作录屏与脚本回放对比等。

四、交付物

- 源代码仓库链接（GitHub 等）。
- `README.md`（含完整说明和示例）。
- 示例文件：
 - 自然语言任务描述（`task.txt`）。
 - 最终生成的独立脚本（`script_final.py`）。
 - 调试修复过程日志（`debug.log`，如有）。
 - 脚本执行成功截图或终端输出。

五、评分标准（共 100 分）

维度	分值	说明
生成模块	25	LLM 指令分解准确，与 DrissionPage 联动流畅，动作捕获完整，初始脚本稳定、可读性好。
Debug 模块	25	能自动检测失败，LLM 修复逻辑有效，修复循环合理，修复日志清晰。
容错模块	20	选择器降级、智能等待、异常处理等策略落地，最终脚本健壮性强。
架构与代码质量	15	三模块解耦，扩展性好，代码规范，日志完善。
文档与演示	15	README 详细易读，示例完整，可快速复现端到端流程。

六、注意事项

- 最终生成的脚本必须是纯 DrissionPage 代码，不包含任何 LLM API 调用或 Agent 特定依赖。
 - 允许在 Debug 模块中使用 LLM 进行脚本修复，但修复后的脚本必须独立运行。
 - 请在 `README` 中说明如何借鉴 AgentBrowser 的生成/Debug/容错架构，并阐述你选择将哪些部分落地到 DrissionPage 生态中的具体方案。
-